Predictive Modeling for Computational Drug Design

E4040.2025Fall.HAHA.report.kk3764.fnz2101.td2849

Ricky Kim kk3764, Nathan Zou fnz2101, Julie Do td2849

Department of Chemical Engineering, Columbia University, New York, NY 10027, USA

Abstract

Predicting critical manufacturing outcomes is essential for optimizing pharmaceutical tablet production, yet remains challenging due to the inherent complexity of process dynamics. In this project, we build on Žagar's work in the "*Big data collection in pharmaceutical manufacturing and its use for product quality predictions*" (2022) [1] by applying machine learning techniques to forecast two key quality metrics, including total process waste and total impurity levels, using batch-level design variables and time-series sensor data. Using DBSCAN-based filtering and correlation analysis, we constructed a reliable feature set, followed by Random Forest and Extreme Gradient Boosting (XGBoost) models to quantify the relative influence of each process variable. These models reveal two distinct regimes: total waste is predominantly driven by fill geometry and mixing-force dynamics, whereas total impurities are driven by diffuse mechanical variability, particularly compression-force fluctuations. To determine whether temporal information improves predictive value, we trained three deep learning (DL) architectures (1D CNN, LSTM, and hybrid CNN+LSTM) on the full multivariate time-series. The 1D CNN achieved the lowest prediction error across both targets, clearly outperforming traditional ML models for total waste, but offering limited improvements for total impurities due to weak temporal signal. Overall, the results show that DL enhances prediction when strong sequential patterns exist, while ML methods remain competitive, interpretable, and robust.

1. Introduction

Technological advancements, particularly in data science, are transforming pharmaceutical manufacturing, enabling the routine collection of high-frequency, multivariate sensor data from modorn tablet-press and continuous-manufacturing systems [1]. Traditional machine learning (ML) approaches offer the ability to predict critical attributes of pharmaceutical products, optimize manufacturing parameters, and reduce experimental costs using per-batch features. However, these traditional ML approaches typically rely on manually engineered, per-batch features derived from raw manufacturing signals, requiring substantial domain expertise and additional data preprocessing steps that can be time-consuming and may limit the ability to fully exploit high-frequency temporal information. The increasing availability of raw, multivariate time-series data in the manufacturing industry motivates the use of deep learning (DL) architectures that can automatically learn process-relevant representations directly from high-frequency manufacturing signals. This can potentially reduce dependence on manual feature engineering and also provide a method for capturing temporal dependence that classical methods might miss.

Early applications of machine learning in process manufacturing typically involved substantial manual feature engineering. Domain experts would transform raw sensor signals into specific descriptors (mean values, variances, peak metrics over time windows) to use as inputs for ML models. Common algorithms included linear or logistic regression, support vector machines (SVM), tree-based methods like Random Forests or gradient boosting (XGBoost), and shallow neural networks. For example, in polymer manufacturing, numerous works built "soft sensor" models for quality variables using methods such as SVMs, Gaussian process regression (GPR), and partial least squares (PLS) regression. These models demonstrated effectiveness on specific tasks, but they often required careful selection and preprocessing of input features. In many cases, temporal information had to be introduced manually (such as by adding lagged variables or time windows). This reveals the a major flaw in statistical descriptors, where they fail to capture crucial information if the signal's trajectory or the order of fluctuations is important for predicting certain outputs.

The emergence of deep learning techniques such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), and more recently, Transformer networks overcomes the requirement of inputting precomputed statistical features.

In this study, we use a publicly available dataset of 1,005 pharmaceutical production batches. Our goal is to model and predict two key manufacturing outcomes, including total process waste (`total_waste`) and total level of impurities (`Total impurities`). We begin with seven domain-selected design variables:

- `tbl_fill_mean`: Mean tablet fill
- `cyl_height_mean`: Mean cylinder height
- `startup_tbl_fill_maxDifference`: Maximum difference of startup tablet fill

- `startup_tbl_fill_mean`: Mean tablet fill during startup
- `main_compForce_mean`: Mean main compression force
- `main_compForce_sd`: Standard deviation of main compression force
- `ejection_mean`: Mean ejection force.

We then systematically expand this initial feature set using a feature selection pipeline to identify additional process variables that are most predictive of `total_waste` and `Total impurities`, yielding an enlarged set of design variables.

Our first modeling approach (Model 1) applies traditional ML regression techniques, including Random Forest and Extreme Gradient Boosting (XGBoost), trained on these pre-aggregated design variables. The extensive preprocessing and data selection workflow to design the traditional ML models is outlined in this report. Model transparency is promoted by utilizing global permutation feature importance analyzes and local interpretability techniques based on Shapley Additive Explanations (SHAP).

Our second approach (Model 2) evaluates three DL architectures: a 1D Convolutional Neural Network (CNN), a Bidirectional Long Short-Term Memory (BiLSTM) network, and a hybrid CNN+BiLSTM model, trained directly on raw multivariate time-series data. These models aim to capture temporal structure and sensor dynamics that are potentially inaccessible to feature engineered ML approaches.

By comparing ML and DL methods under shared evaluation metrics, we assess predictive accuracy, generalization, interpretability, computational cost, and vulnerability to overfitting. Together, these analyzes provide a comprehensive evaluation of when traditional feature-based models suffice and when DL offers clear advantages for data-rich pharmaceutical manufacturing.

2. Dataset

The dataset used in this study is the "*Cholesterol-Lowering Drug Process and Quality Data*" (2021) [2]. It consists of 1,005 production batches collected from a commercial pharmaceutical tablet-manufacturing process. The dataset contains two primary files:

- **Process.csv**: Provides batch-level and sensor-level process data, including tablet-press operational variables (e.g., `tbl_speed_mean`, `main_CompForce_mean`, `tbl_fill_mean`), batch performance metrics (e.g., `total_waste`), and laboratory quality measurements (e.g., `Total impurities`).
- **Normalization.csv**: Contains batch size information and normalization factors for each product code, enabling adjustment for differences in production scale across batches.

In addition to these files, the dataset includes 25 raw time-series files containing high-frequency sensor measurements captured during tablet production. These files provide trace-level data, such as `tbl_speed`, `fom`, `main_comp`, `tbl_fill`, `SREL`, `pre_comp`, `produced`, `waste`, `cyl_main`, `cyl_pre`, `stiffness`, and `ejection`, sampled continuously throughout each batch. After loading, the time-series are linked to batch identifiers and used for DL models.

Together, these files provide multivariate information that includes raw time-series sensor traces, aggregated process statistics, and final quality outcomes. The dataset, therefore, supports both traditional ML approaches and time-series DL modeling for batch-level predictions.

3. Methods

3.1. Machine Learning Pipeline

3.1.1. Data Preprocessing

Before modeling, a series of data preprocessing steps were applied to ensure reliable model performance. The file `Process.csv` was first merged with `Normalization.csv` using the product code to account for batch size variability. All batch-dependent variables were normalized by their corresponding batch-size factor:

$$\text{Normalized Value} = \frac{\text{Raw Value}}{\text{Normalization Factor}} \tag{1}$$

where the normalization factor corresponds to batch size (e.g., a product code with 240,000 tablets has a normalization factor of 2.4). Rows with missing values were removed after merging and normalization to ensure consistent input across all models. The batch-dependent variables include:

- `total_waste`
- `tbl_speed_0_duration`
- `tbl_speed_change`
- `tbl_fill_mean`
- `cyl_height_mean`
- `Startup_tbl_fill_maxDifference`
- `Startup_tbl_fill_mean`
- `startup_waste`
- `fom_change`.

Design variables were defined as all process features excluding:

- Identifier columns (`batch`, `code`, `Product code`, `Normalisation factor`, `weekend`)
- Redundant max/min statistics (`ejection_max`, `ejection_min`, `stiffness_max`, `stiffness_min`, `SREL_production_max`)

- Variables directly tied to the targets (`startup_waste`)
- Laboratory quality-control outputs (`Drug release average (%)`, `Drug release min (%)`, `Residual solvent`, `Impurity O`, `Impurity L`)
- Two targets (`total_waste`, `Total impurities`).

All design variables were then standardized using `StandardScaler`. Standardization was necessary because both Density-Based Spatial Clustering of Applications with Noise (DBSCAN) and Principal Component Analysis (PCA) rely on Euclidean distances, which preserve the Gaussian distribution of the original variables (e.g., mean-centered, unit-variance features).

Outlier detection was performed using DBSCAN, an unsupervised learning algorithm capable of identifying samples in sparse regions as noise points. Unlike traditional clustering methods, DBSCAN can detect clusters of arbitrary shapes and automatically label outliers without requiring prior specification of the number of clusters [3]. Samples labeled as noise (`dbscan_label = -1`) were removed, and all subsequent correlation and regression analyses were performed on the remaining 534 batches.

Two DBSCAN hyperparameters were tuned:

- `min_samples`: the minimum number of points required to form a cluster. It was initially estimated using the heuristic $2 \times$ dimensions [4]. A practical range of 5 to 25 was also tested, balancing overfitting (too low) and underfitting (too high).
- `eps`: the maximum neighborhood radius. It was varied from 0.5 to 1.8 after standardizing the data.

For each parameter combination, clustering quality was evaluated using the silhouette score, considering only configurations that produced more than one non-noise cluster. The silhouette score measures how well a point fits within its assigned cluster relative to other clusters, balancing cohesion (similarity or proximity to points in the same cluster) and separation (distance from points in the nearest other cluster). The silhouette score is defined as

$$\text{Silhouette} = \frac{b - a}{\max(a, b)} \tag{2}$$

where a is the average intra-cluster distance and b is the average nearest-cluster distance. Higher silhouette scores indicate more internally consistent and well-separated clusters, while low or negative scores suggest overlapping or misclassified clusters. A silhouette score above 0.5 is generally considered strong.

For visualization only, principal component analysis (PCA) was applied to the standardized design variables, and the first two principal components (PC1, PC2) were used to plot the DBSCAN clusters in two dimensions. The first principal component (PC1) captured the maximum variance direction, while the second (PC2) captured the second most, orthogonal to the first [5].

3.1.2. Design Variable Selection

To systematically expand beyond the initial seven domain-selected variables, Pearson correlations between all remaining process variables and each target were computed on the DBSCAN-filtered dataset. Variables showing meaningful correlations were retained as candidates for nonlinear modeling.

3.1.3. Predictive Modeling Framework

After preprocessing, the dataset was split into training and testing subsets using an 80/20 split ratio. Separate Random Forest and XGBoost regressors were trained for each target variable. Model performance was assessed using the coefficient of determination (R^2).

Random Forest Regression. Random Forest was utilized for its ability to capture nonlinearities and handle feature interactions. We selected hyperparameters from typical ranges used for tabular regression tasks, and in the final models used:

- `n_estimators` = 200 – Number of trees; higher values can improve performance at the cost of computation.
- `max_depth`: None – Maximum depth of each tree; deeper trees capture complex patterns but risk overfitting.
- `min_samples_split` = 2 – Minimum samples required to split a node; higher values add regularization.
- `min_samples_leaf` = 1 – Minimum samples at a leaf node; larger values reduce variance.
- `max_features` = `sqrt` – Number of features considered for each split; using `sqrt` adds randomness and reduces overfitting. The option `log2` was omitted, as its behavior closely resembles `sqrt` for small feature spaces.

XGBoost. XGBoost, an efficient implementation of gradient boosting, was used to model complex feature interactions and nonlinear patterns in the data. The algorithm offers fine-grained control over model complexity and regularization, and it is also robust to missing values and variations in feature scale [6]. With more refined boosting strategy, XGBoost often outperforms Random Forest on tabular datasets. Hyperparameters used were:

- `n_estimators` = 200 – Number of boosting rounds; higher values increase model capacity but risk overfitting.
- `max_depth` = 5 – Maximum tree depth; deeper trees model more complex relationships but also at the cost of increased variance.
- `learning_rate` = 0.1 – Step size shrinkage; lower values improve generalization.
- `colsample_bytree` = 1.0 – Fraction of features used per tree; reduces overfitting risk by decorrelating trees.
- `subsample` = 1.0 – Fraction of training data used per tree; set to 1.0 to use the full dataset in each boosting round, avoiding additional stochasticity.

- `reg_alpha` = 0 – L1 regularization was disabled to avoid enforcing feature sparsity and focus more on capturing all relevant feature interactions.

- `reg_lambda` = 1 – Moderate L2 regularization was applied to penalize large weights and reduce overfitting risk.

3.1.4. Model Interpretation Strategies

To better understand how the models made their predictions, both global and local model interpretation techniques were used.

Global Interpretation. Global interpretation methods help explain how models behave across the entire dataset by identifying which features contribute most to predictions. Two main strategies were utilized:

- **Feature Importance**: Scores were extracted from the model via the `feature_importances_` attribute, which reflects how frequently and effectively each feature is used to split the data in the ensemble.

- **Permutation Importance**: Scores were computed by randomly shuffling each feature in the test set and measuring the resulting drop in model performance. Features that caused large decreases in R^2 were considered more important.

Local Interpretation. Local interpretation techniques were employed to explain individual model predictions, highlighting how specific feature values influenced model outputs. SHAP values, based on cooperative game theory, assign a consistent contribution score to each feature for a given prediction. SHAP reveals both the magnitude and direction of each feature's effect relative to the model's average output. Unlike simple sensitivity analyses, SHAP also accounts for feature interactions and ensures additivity in its attributions [7].

Explanations were generated using `TreeExplainer` as it is specifically optimized for tree-based models such as Random Forest and XGBoost. It provides fast runtime and exact SHAP values, whereas the general explainer relies on approximations and can be computationally expensive.

Therefore, for each model and target, we computed (i) intrinsic feature importance, (ii) permutation importance, and (iii) SHAP values. Features that were consistently influential across both models and all importance measures were retained as candidates for the final design set. This procedure yielded a screened set of 11 design variables, including `cyl_height_mean`, `tbl_fill_mean`, `fom_change`, `tbl_speed_change`, `Startup_tbl_fill_mean`, `tbl_speed_0_duration`, `fom_mean`, `main_CompForce_mean`, `main_CompForce_sd`, `Startup_tbl_fill_maxDifference`, and `ejection_mean`. These features were passed forward as candidate inputs for subsequent DL modeling.

3.2. Deep Learning Network

3.2.1. Data Preprocessing

We implemented a structured preprocessing pipeline to convert multivariate sensor streams and batch-level process data into fixed-length sequences suitable for DL models. Throughout, we adopt the Keras-style tensor notation (B, T, C), where B is the number of batches, T is the standardized sequence length, and C is the number of input channels.

From `Process.csv`, we extracted the batch identifier and the two prediction targets (`total_waste` and `Total impurities`). Only batches with valid target values were retained to ensure reliable supervision. Both targets were then standardized using a `StandardScaler` to obtain zero-mean, unit-variance supervision signals.

All available time-series CSV files (25 total in our dataset) were concatenated into a single multivariate dataset containing approximately 4.7 million rows. From these files, we selected the 12 sensor channels used in the DL models: `tbl_speed`, `fom`, `main_comp`, `tbl_fill`, `SREL`, `pre_comp`, `produced`, `waste`, `cyl_main`, `cyl_pre`, `stiffness`, `ejection`.

Two of the channels, including `produced` and `waste`, represent batch size dependent counts. These were normalized by dividing by the respective batch specific normalization factor from `Normalization.csv`, consistent with the earlier ML batch level preprocessing steps. The remaining variables are metrics related to the manufacturing equipment and geometry, which are batch-size independent and therefore do not need to be normalized by the batch-specific normalization factors.

Because sensor streams contain occasional gaps, each batch sequence was imputed using forward filling, backward filling, and replacing any residual missing entries with zeros. This ensured continuity within each sequence and avoided interruptions in temporal patterns.

Because batch durations varied widely, sequence lengths were standardized before modeling. The 95th percentile of batch sequence lengths was computed and capped at a maximum allowable sequence length at 5000 time steps. For each batch, sequences longer than this limit were truncated, and shorter sequences were zero-padded.

Layer	Details	Output	Params
Input	Multivariate time series	*None* × 5000×12	0
Conv-1	32 filters, kernel size = 5 stride = 1 padding = `same` L2 regularization, no bias	*None* × 5000×32	1,920
Norm-1	32 channels	*None* × 5000×32	64
ReLU-1	Activation	*None* × 5000×32	0
Dropout-1	$p = 0.1$	*None* × 5000×32	0
Conv-2	64 filters, kernel size = 5 stride = 1 padding = `same` L2 regularization, no bias	*None* × 5000×64	10,240
Norm-2	64 channels	*None* × 5000×64	128
ReLU-2	Activation	*None* × 5000×64	0
Dropout-2	$p = 0.1$	*None* × 5000×64	0
Global Avg Pool	GlobalAveragePooling1D	*None* × 64	0
Dense-1	units = 64 activation = `relu` L2 regularization	*None* × 64	4,160
Dropout-3	$p = 0.2$	*None* × 64	0
Dense-2	units = 32 activation = `relu` L2 regularization	*None* × 32	2,080
Output Dense	units = 2 activation = `linear`	*None* × 2	66
Total parameters			**19,658**

Table 1: Layer-by-layer 1D CNN architecture used for the DL network.

To ensure each sensor contributed proportionally during training, the time-series tensor was flattened along the batch and time dimensions. Each channel was then standardized using `StandardScaler`, and the result was reshaped back into (B, T, C) format. This approach preserves cross-channel comparability even when raw units differ.

Finally, data were split at the batch level into an 80/20 train-validation split. The result is a standardized and uniformly structured tensor dataset ready for input into CNN, BiLSTM, and hybrid CNN–BiLSTM models evaluated in our study.

Layer	Details	Output	Params
Input	Multivariate time series	*None* × 5000×12	0
BiLSTM-1	units = 64 return_sequences = True L2 regularization dropout = 0.3	*None* × 5000 × 128	39,424
BiLSTM-2	units = 32 return_sequences = False L2 regularization dropout = 0.3	*None* × 64	41,216
Dense-1	units = 64 activation = ReLU use_bias = False L2 regularization	*None* × 64	4,096
ReLU-1	Activation	*None* × 64	0
Dropout-1	$p = 0.5$	*None* × 64	0
Dense-2	units = 32 activation = ReLU use_bias = False L2 regularization	*None* × 32	2,048
ReLU-2	Activation	*None* × 32	0
Dropout-2	$p = 0.5$	*None* × 32	0
Output Dense	units = 2 activation = linear	*None* × 2	66
Total parameters			**86,850**

Table 2: Layer-by-layer Bidirectional LSTM architecture for the DL network.

3.2.2. Software Design

The file `DL_model.ipynb` implements the full DL pipeline for predicting tablet-manufacturing quality outcomes from multivariate time-series data. The notebook interfaces with three model-definition modules located in `utils/`: `cnn.py`, `bilstm.py`, and `cnn_bilstm.py`. Each module defines a network architecture together with a trainer class that manages optimization, checkpointing, and learning-rate scheduling. Across all models, the trainers use a shared training framework that includes:

- Adam optimization
- L2 weight regularization
- cosine learning-rate decay
- MAE as the validation metric
- early stopping based on validation MAE
- saving the best-performing model weights

Layer	Details	Output	Params
Input	Multivariate time series	*None* × 5000×12	0
Conv-1	32 filters, kernel size = 5 stride = 1, padding = same use_bias = False, L2 reg	*None* × 5000×32	1,920
Norm-1	32 channels	*None* × 5000×32	64
ReLU-1	Activation	*None* × 5000×32	0
Dropout-1	$p = 0.1$	*None* × 5000×32	0
Conv-2	64 filters, kernel size = 5 stride = 1, padding = same use_bias = False, L2 reg	*None* × 5000×64	10,240
Norm-2	64 channels	*None* × 5000×64	128
ReLU-2	Activation	*None* × 5000×64	0
Dropout-2	$p = 0.1$	*None* × 5000×64	0
BiLSTM	units = 64 return_sequences = False bidirectional, L2 reg, dropout	*None* × 128	66,048
Dense-1	units = 64 activation = ReLU use_bias = False	*None* × 64	8,192
ReLU-3	Activation	*None* × 64	0
Dropout-3	$p = 0.2$	*None* × 64	0
Dense-2	units = 32 activation = ReLU use_bias = False	*None* × 32	2,048
ReLU-4	Activation	*None* × 32	0
Output Dense	units = 2 activation = linear	*None* × 2	66
Total parameters			**88,706**

Table 3: Layer-by-layer CNN+BiLSTM architecture for the DL network.

The notebook concludes with model evaluation: training/validation curve plots, predicted-vs-actual comparisons, and residual-error distributions for each target. This pipeline therefore includes the data processing, training, and evaluation workflow for three architectures: a 1D CNN, a BiLSTM, and a hybrid CNN-BiLSTM network.

`utils/cnn.py` implements a 1D CNN model through the `CNN_trainer` class. The architecture, summarized in Table 1, consists of two Conv1D–BatchNorm–ReLU–Dropout blocks followed by global average pooling and a small dense regression head. Training incorporates additive Gaussian noise augmentation.

`utils/bilstm.py` defines a pure recurrent baseline via the `BiLSTM_trainer` class. As shown in Table 2, the architecture uses two stacked bidirectional LSTM layers (sequence-to-sequence followed by sequence-to-vector) to capture long-range temporal dependencies with L2 regularization on both kernel and recurrent weights. During training, additive Gaussian noise is applied for augmentation.

`utils/cnn_bilstm.py` provides a hybrid model through the `CNN_BiLSTM_trainer` class, combining convolutional and recurrent architectures for multivariate time-series regression, as summarized in Table 3. Initial 1D convolutional layers extract local short-range temporal patterns and reduce input complexity, which are then processed by a bidirectional LSTM layer to capture long-term dependencies in both past and future directions. During training, Gaussian noise is applied for data augmentation.

3.2.3. Training Algorithm Details

All three models were trained under a shared training scheme for consistency and comparability. Training used the Adam optimizer with an initial learning rate of 1×10^{-3} and a cosine decay. Models were trained for up to 100 epochs with a mini-batch size of 128.

To improve robustness to sensor noise, Gaussian noise augmentation was applied during training: each input sequence was perturbed with zero-mean noise scaled to 5% of the global feature standard deviation. Regularization was matched to architecture: convolutional layers used dropout with $p = 0.1$, dense layers used $p = 0.2$, and all learnable kernels included L2 weight penalties with coefficient 1×10^{-5}.

Early stopping with a patience of five epochs prevented overfitting, and the best model parameters (based on validation MAE) were restored automatically.

4. Results

4.1. Traditional ML Results

4.1.1. DBSCAN Clustering and Outlier Removal

Prior to applying DBSCAN for clustering and outlier detection, all batch-dependent variables were normalized using the normalization factors described above, and the resulting design variables were standardized. After normalization of batch-dependent variables and removal of rows containing missing values, the dataset was reduced from 1,005 to 987 batches, and there were 19 candidate design variables.

DBSCAN was then applied to the standardized design variables, and silhouette scores were computed across a grid of `eps` and `min_samples` hyperparameters. The best-performing configuration was `eps` = 1.8 and `min_samples` = 20, yielding an average silhouette score of 0.158. Although this value indicates only weak cluster structure, it is still positive and sufficient for identifying dense process regions versus sparse outliers.

Retraining DBSCAN with the optimal parameters produced two process clusters (labels 0 and 1) along with a group of outliers labeled as −1, as shown in **Figure 1**. The clusters were

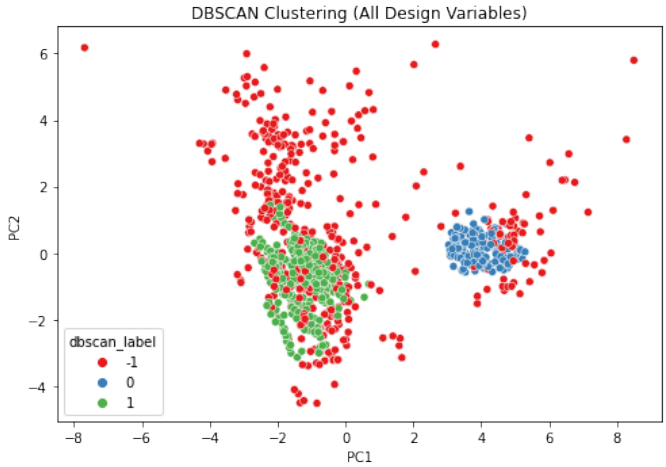

Figure 1: DBSCAN clustering results visualized in PCA space (PC1 vs. PC2) using standardized design variables.

visualized in PCA space (PC1–PC2), confirming that both clusters were reasonably separated, while outliers appeared in low-density regions. Samples labeled as -1 were removed to create a cleaner dataset (534 batches) for downstream regression modeling. A silhouette plot was generated using only the retained samples (Appendix, **Figure 14**). Both clusters exhibited predominantly positive silhouette values with no substantial negative regions. Cluster 0 appeared more compact and internally consistent, whereas Cluster 1 was larger but still cohesive, supporting their use in subsequent modeling.

4.1.2. Design Variable Selection

To identify the most relevant design variables beyond the initially selected seven, correlations between all candidate process variables and the targets (`total_waste` and `Total impurities`) were computed using the DBSCAN-filtered dataset. Variables showing meaningful correlations were retained for nonlinear modeling.

For `total_waste`, the strongest positive correlations among non-target process variables were observed for:

- `tbl_speed_change`
- `fom_change`
- `tbl_fill_mean`
- `tbl_speed_0_duration`
- `Startup_tbl_fill_mean`
- `cyl_height_mean`
- `Startup_tbl_fill_maxDifference`
- `fom_mean`
- `main_CompForce_mean`
- `main_CompForce_sd`.

Additionally, two quality-control outputs (`Drug release average (%)` and `Drug release min (%)`) exhibited moderate correlations with `total_waste` and were retained to capture dissolution process.

For `Total impurities`, the highest correlations among non-leakage variables were observed for:

- `main_CompForce_sd`
- `cyl_height_mean`
- `tbl_fill_mean`
- `Startup_tbl_fill_mean`
- `ejection_mean`
- `main_CompForce_mean`
- `fom_mean`
- `tbl_speed_change`
- `tbl_speed_0_duration`.

Combining these variables produced a set of 14 candidate predictors for subsequent nonlinear modeling. These features were then used as inputs to Random Forest and XGBoost models to quantify feature importance and guide final design variable selection for DL approaches.

Selected features for downstream modeling (from correlation and interpretation analysis): `tbl_speed_change`, `fom_change`, `tbl_fill_mean`, `tbl_speed_0_duration`, `Startup_tbl_fill_mean`, `cyl_height_mean`, `Startup_tbl_fill_maxDifference`, `fom_mean`, `main_CompForce_mean`, `main_CompForce_sd`, `ejection_mean`, `tbl_fill_sd`, `Drug release average (%)`, `Drug release min (%)`.

4.1.3. Predictive Modeling Results

Table 4 summarizes the hold-out R^2 scores for Random Forest and XGBoost using the screened design variables. XGBoost achieved a slightly higher R^2 for `total_waste`, whereas Random Forest performed substantially better for `Total impurities`, yielding a more balanced overall fit across the two targets. Although XGBoost achieved a slightly higher R^2 on `total_waste`, its performance on `Total impurities` was substantially lower than Random Forest. Because Random Forest provides more balanced accuracy across both targets and yields more stable SHAP attributions, it was selected as the primary model for global and local feature importance interpretation.

Model	**Test R^2** `total_waste`	**Test R^2** `Total impurities`
Random Forest	0.6439	0.5801
XGBoost	0.6919	0.4505

Table 4: Predictive model performance for `total_waste` and `Total impurities`.

4.1.4. *Interpretation of Predictive Models*

Feature importance analysis was performed using both Random Forest and XGBoost to identify which design variables most strongly influence the manufacturing outcomes. Although XGBoost achieved slightly higher predictive performance for `total_waste`, Random Forest was used for SHAP interpretation because SHAP explanations are substantially more stable and less sensitive to hyperparameters. All SHAP plots, therefore, reflect Random Forest fits.

Total Waste.. Across both Random Forest and XGBoost, three variables emerged as the dominant predictors:

- `cyl_height_mean`
- `tbl_fill_mean`
- `fom_change`.

In the Random Forest model, `cyl_height_mean` had the highest feature importance (Figure 2) and produced the largest drop in R^2 under permutation (Figure 3), indicating that increases in the average cylinder height strongly elevate predicted waste. XGBoost (Appendix Figures 15, 16) ranked `tbl_fill_mean` among the most influential variables. `fom_change` and `tbl_speed_change` were also consistently influential, showing a strong importance in both Random Forest and XGBoost. SHAP analysis further clarifies directional effects (Figure 4). Higher values of `cyl_height_mean` and `tbl_fill_mean` generally increase the predicted total waste. `fom_change` displays substantial horizontal dispersion in the SHAP plot, indicating that fluctuations in the force-of-mixing have heterogeneous effects across batches. Other features such as `tbl_speed_change`, `main_CompForce_sd`, `Startup_tbl_fill_mean`, and `tbl_speed_0_duration` contributed modestly but consistently across Random Forest permutation importance and SHAP.

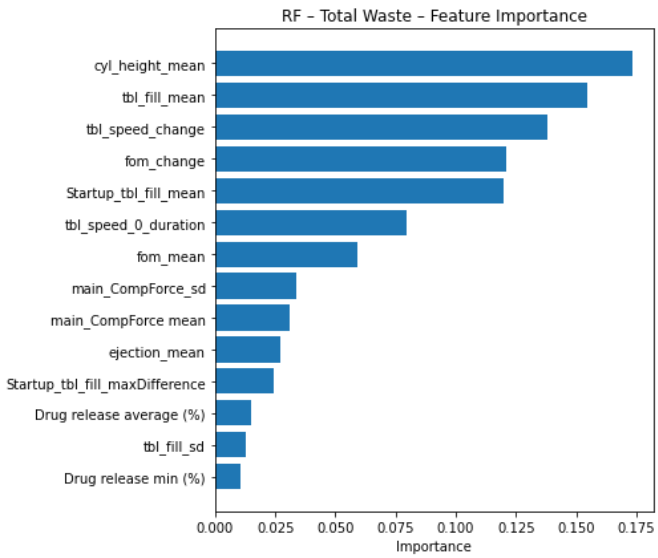

Figure 2: Feature importance for `total_waste` using Random Forest.

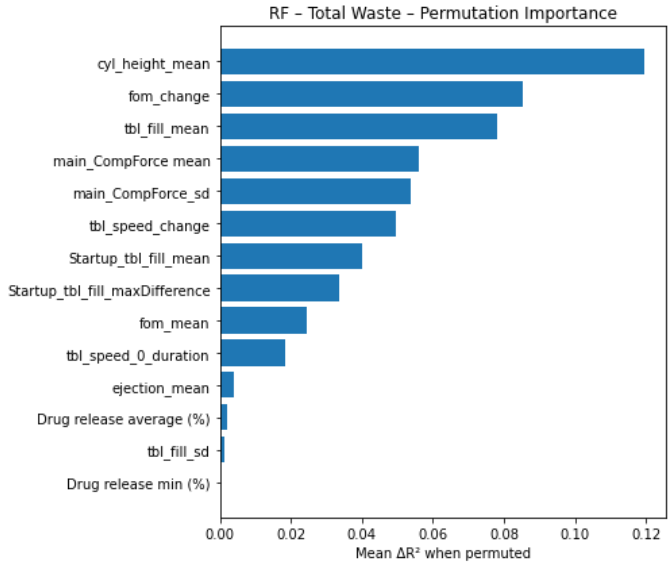

Figure 3: Permutation importance for `total_waste` using Random Forest.

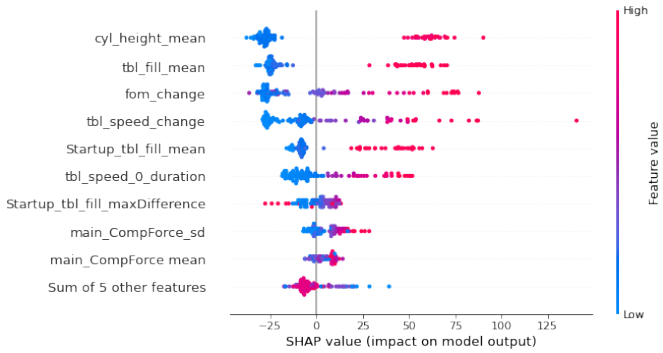

Figure 4: SHAP plot for `total_waste` using Random Forest.

Furthermore, the XGBoost SHAP plot (Appendix Figures 17) shows that `tbl_fill_mean`, `fom_change`, and `cyl_height_mean` are also the dominant drivers of predicted waste. `tbl_speed_change` also exhibits large SHAP variability. Startup variables such as `Startup_tbl_fill_mean` and `Startup_tbl_fill_maxDifference` contribute moderately, while lower ranked features cluster near zero, reflecting limited marginal influence. Overall, the XGBoost model attributes waste primarily to fill depth, height variation, and mixing-force dynamics.

Total Impurities. Although overall predictive performance for impurity levels was modest (Table 4), the feature importance profiles were stable across Random Forest and XGBoost. Across both algorithms, `cyl_height_mean` and `main_CompForce_sd` were the strongest predictor (see **Figure 5** and **Figure 6**), confirming that the total impurity is primarily driven by tablet height and compression force variability.

SHAP values indicate that higher `tbl_fill_mean` tends to reduce the predicted impurities, while higher compression force variability (`main_CompForce_sd`) generally increases the impurity levels, as shown in **Figure 7**. This suggests that mechan-

ical inconsistency during compaction may affect total impurities. `ejection_mean` exhibits a mix of positive and negative contributions. The fill-related variables (`tbl_fill_mean` and `Startup_tbl_fill_mean`) show a weaker influence and narrower SHAP spreads—consistent with their mid rank positions in feature importance plots.

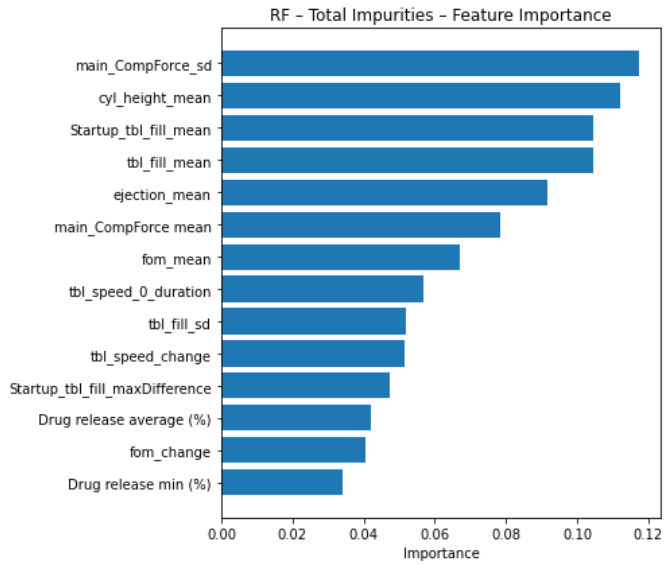

Figure 5: Feature importance for `Total impurities` using Random Forest.

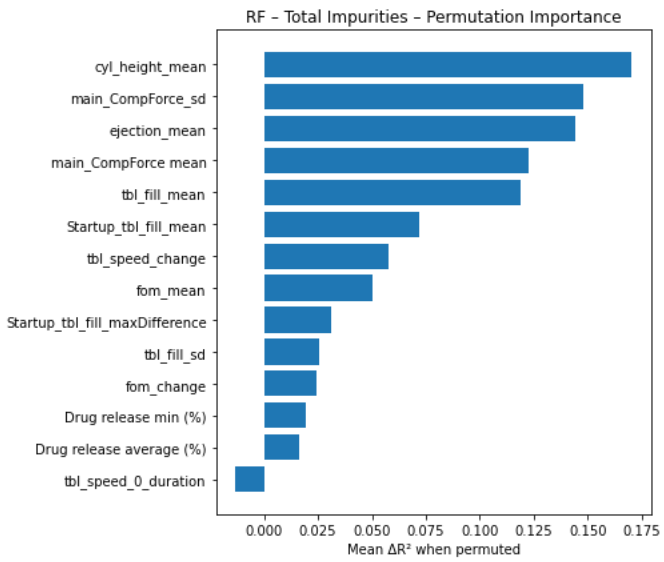

Figure 6: Permutation importance for `Total impurities` using Random Forest.

In contrast to the waste model, no single feature overwhelmingly dominates impurity prediction because importance values gradually decrease, and SHAP spreads are relatively narrow. This behavior aligns with the weak correlations observed earlier and suggests that total impurities is governed by multiple factors rather than one or two leading drivers.

The XGBoost model (Appendix Figures 18, Figures 19, Figures 20) produced feature importance patterns that closely

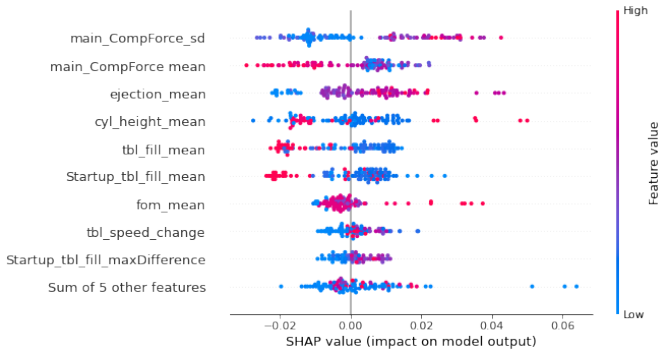

Figure 7: SHAP plot for `Total impurities` using Random Forest.

mirrored those from Random Forest. In both algorithms, `main_CompForce_sd` and `main_CompForce_mean` remained the two dominant drivers, with XGBoost additionally elevating `ejection_mean` and `cyl_height_mean` as influential secondary factors. The permutation importance plot shows a similarly gradual decline across features, indicating that impurity prediction depends on a distributed set of parameters rather than a single leading driver. SHAP values reinforce this pattern, where the spreads are relatively narrow compared to the waste model, and no feature exhibits the large, asymmetric influence seen for waste-related variables. Overall, XGBoost supports the Random Forest conclusion that total impurities is governed by mechanical stability and compression force consistency, but it places greater emphasis on ejection behavior.

Overall, the combined evidence from Random Forest, XGBoost, permutation importance, and SHAP analysis reveals two distinct mechanistic regimes: total waste is driven primarily by fill geometry and mixing-force dynamics, while total impurities levels arise from more diffuse influences related to mechanical stability and compression force variability. These insights directly informed the selection of design variables for the DL models.

4.2. DL Results

4.2.1. Data Processing

Only batches with valid target values were retained, resulting in 987 usable batches. Time-series rows corresponding to these batches were preserved, and missing feature values within each batch were imputed using forward-fill, backward-fill, and remaining NaNs filled with zero. Sequence lengths varied across batches, so sequences were truncated or zero-padded to a maximum length of 5,000 timesteps, producing a consistent tensor: $X \in \mathbb{R}^{987\times5000\times12}$, $y \in \mathbb{R}^{987\times2}$.

All features were standardized per channel across all timesteps and batches using a `StandardScaler`. Target variables were similarly standardized. Finally, the dataset was split at the batch level into an 80/20 train–validation split: X_{train} : 789 batches, X_{val} : 198 batches.

4.2.2. MAE Loss

Figure 8 compares the learning curves of the three architectures. The 1D CNN shows the smoothest and most stable con-

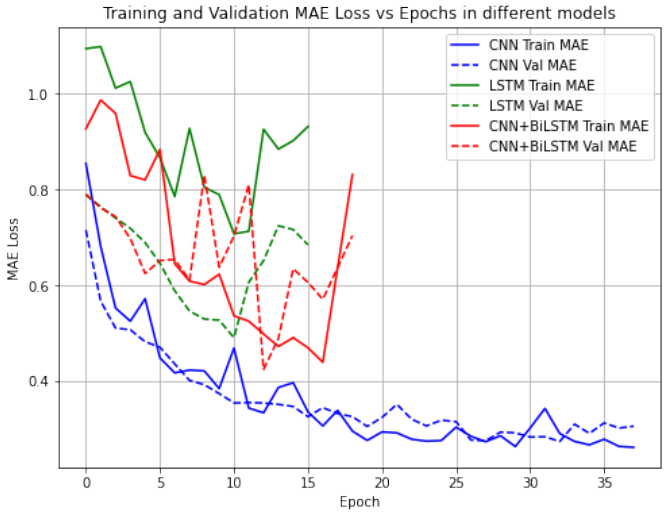

Figure 8: Training and validation MAE across epochs for all models.

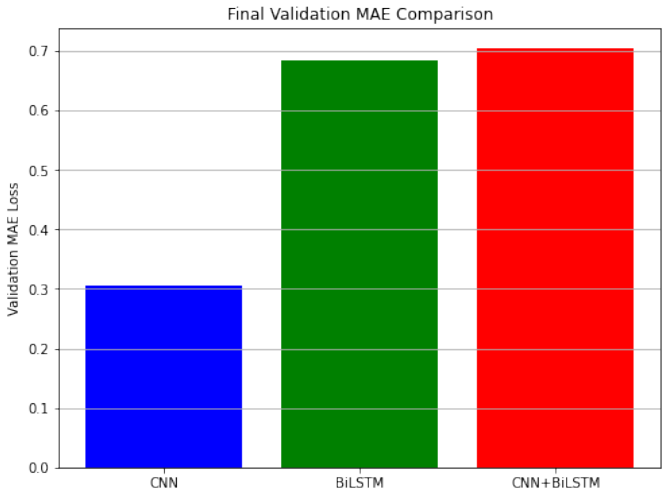

Figure 9: Final validation MAE comparison across all architectures.

vergence. Both training and validation MAE decrease steadily and remain closely aligned, ultimately reaching the lowest error among all models. In contrast, the BiLSTM converges more slowly and exhibits a noisier validation trajectory, with higher MAE and larger fluctuations across epochs. The hybrid CNN+BiLSTM model initially reduces validation MAE but soon develops a persistent gap between training and validation curves, indicating that it overfits the training set without gaining corresponding generalization improvements.

Figure 9 summarizes the final validation MAE values. CNN achieves the lowest error (approximately 0.3), confirming that purely convolutional temporal feature extraction is the most effective among the tested architectures for this dataset. The BiLSTM attains substantially higher validation MAE (around 0.7), consistent with its noisier and slower convergence. The hybrid CNN+BiLSTM performs similarly poorly, yielding the highest MAE overall, suggesting that adding recurrent layers on top of the CNN does not improve performance and instead introduces extra complexity that the current amount of data and tuning cannot fully exploit.

4.2.3. Predicted vs. True Outputs

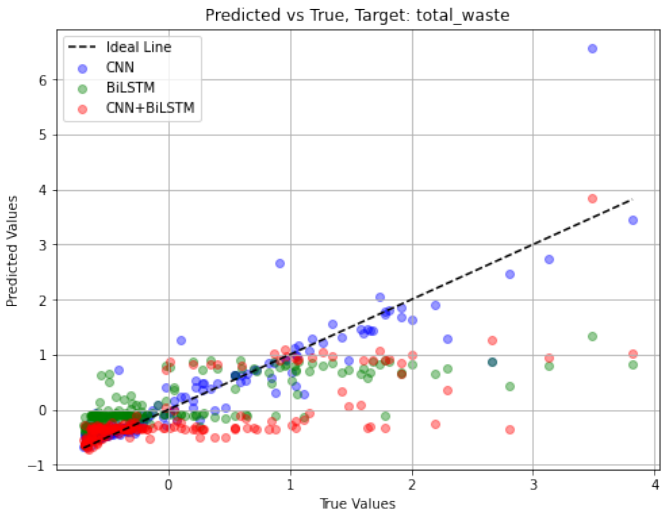

Figure 10: Predicted vs. true values for `total_waste`.

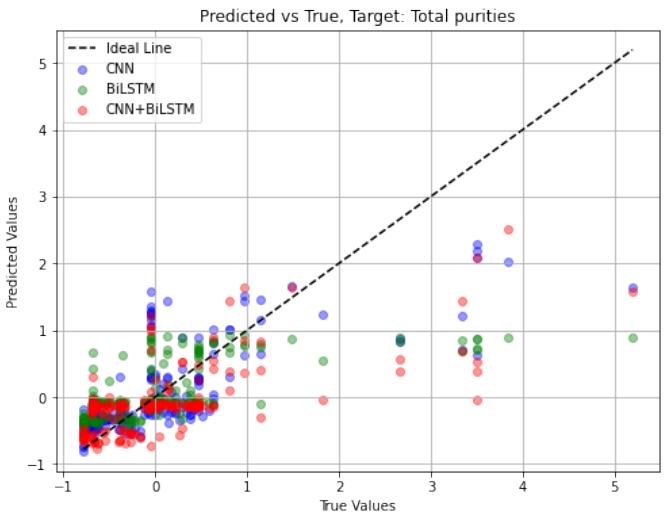

Figure 11: Predicted vs. true values for `Total impurities`.

Figures 10 and 11 compare the predicted and true values for both target variables. For `total_waste`, the CNN predictions cluster closely around the diagonal line, indicating good calibration and relatively unbiased performance across the range of observed values. In contrast, the BiLSTM displays substantially larger scatter and a tendency to underestimate higher waste values, consistent with its noisier learning dynamics observed earlier. The hybrid CNN+BiLSTM model performs the worst for this target: its predictions are tightly compressed near zero and rarely follow the true values. This collapse suggests that the model overfits early but fails to learn a stable mapping for higher outputs.

The effect is even more pronounced for `Total impurities`. Both the CNN and BiLSTM capture qualitative trends but underestimate large total impurity values, reflecting the sparsity of high impurity samples in the data. Meanwhile, the hybrid model again collapses to a narrow range of predictions, capturing almost none of the true variability.

4.2.4. Residual Error Analysis

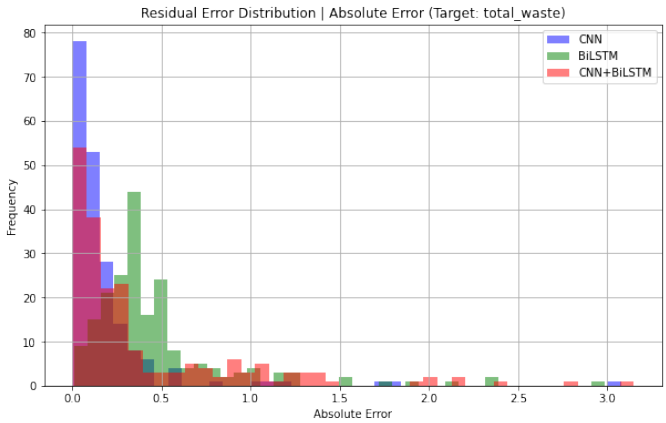

Figure 12: Absolute residual error distribution for `total_waste`.

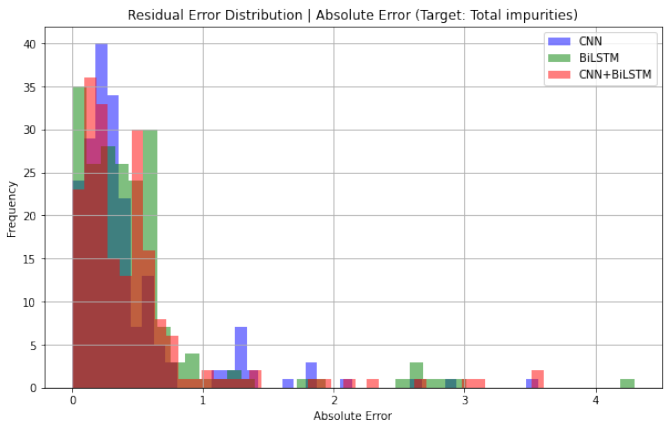

Figure 13: Absolute residual error distribution for `Total impurities`.

Figures 12 and 13 present the absolute residual error distributions for both targets, providing each model's reliability. For `total_waste`, CNN produces the most compact distribution, with the majority of errors concentrated below 0.3. This narrow shape indicates stable predictions across samples and a low frequency of large deviations. The BiLSTM exhibits a noticeably broader spread with a heavier right tail, reflecting its tendency to generate occasionally large errors. The hybrid CNN+BiLSTM model shows an even wider distribution and frequent errors above 1.0, consistent with the underestimation pattern observed earlier.

For `Total impurities`, the contrast among models becomes more pronounced. While CNN again yields the tightest residuals, both the BiLSTM and the hybrid model show substantial error mass extending toward 1.0 and beyond. The hybrid model displays a distinctive accumulation of residuals near zero combined with a long tail of high error outliers, where the model outputs a narrow range of values regardless of the true target.

4.2.5. Computational Cost

Training time per epoch reflects the computational differences between the architectures, as shown in Figure 8. The BiLSTM and CNN+BiLSTM models require noticeably more time per epoch than the CNN due to the sequential recurrence operations in the BiLSTM layers. By contrast, the CNN processes the full 5,000-step multivariate sequences in parallel via convolutions, leading to the fastest epochs and allowing it to reach its best validation MAE in fewer total training steps. In contrast, the LSTM and CNN+LSTM models took longer to train and showed slower improvement across epochs, reflecting their higher computational cost and parameter count.

The three DL architectures differ substantially in their number of parameters and, consequently, their computational costs. The 1D CNN model (Table 1) has the fewest parameters at 19,658, reflecting its lightweight design with only convolutional and dense layers. The bidirectional LSTM (Table 2) has 86,850 parameters, over four times larger than the CNN, primarily due to the recurrent weights required to capture temporal dependencies in both forward and backward directions. The hybrid CNN+BiLSTM model (Table 3) has the highest parameter count at 88,706, combining convolutional feature extraction with bidirectional LSTM processing, which slightly increases model complexity relative to the standalone BiLSTM.

5. Discussion

This study compared traditional ML models (Random Forest and XGBoost) with three DL architectures (1D CNN, LSTM, and a CNN+LSTM hybrid) to evaluate their ability to predict two key pharmaceutical manufacturing outcomes: total waste and total impurities.

5.1. ML Model

DBSCAN-based filtering reduced the dataset from 1,005 to 534 batches by removing sparsely distributed operating conditions. Although the silhouette score indicated only weak cluster separation, the filtering step effectively removed noisy, atypical runs and improved the stability of subsequent regression models.

For `total_waste`, both Random Forest and XGBoost achieved moderate predictive accuracy, with XGBoost performing slightly better (R^2 = 0.6919 vs. 0.6439). Despite this difference, the two models agreed on the dominant predictors:

- `cyl_height_mean`
- `tbl_fill_mean`
- `fom_change`
- `tbl_speed_change`.

These variables ranked highest in feature importance analyzes and exhibited large SHAP spreads, indicating strong and heterogeneous effects across batches. Together, they describe geometric and mixing-related mechanisms, such as fill depth, tablet height, and force-of-mixing fluctuations, that plausibly drive material inefficiency.

In contrast, predictive performance for `Total impurities` was noticeably lower (Random Forest: $R^2 = 0.5801$; XGBoost: $R^2 = 0.4505$). Feature importance analyzes were similar across the two models, but unlike total waste, no single feature dominated. The top predictors were:

- `main_CompForce_sd`
- `main_CompForce_mean`
- `cyl_height_mean`
- `ejection_mean`.

SHAP values exhibited narrow spreads, indicating weaker and more distributed effects. This pattern, together with the low correlation strength observed earlier, suggests that total impurities arises from multiple subtle mechanical interactions rather than a single key driver. The limited R^2 further implies that total impurities may depend on additional unmeasured chemical or environmental factors not captured in the process-level dataset.

Although XGBoost performed slightly better for total waste, Random Forest produced substantially better total impurities predictions and yielded more stable SHAP explanations. Because the study aims to model both outcomes simultaneously and extract interpretable mechanistic insights, Random Forest represents the more balanced and reliable modeling choice.

5.2. DL Model

Across all architectures, the 1D CNN achieved the lowest training and validation MAE for both targets, together with the narrowest residual error distributions. Its smooth learning curves and tight clustering around the ideal predicted–true diagonal line indicate stable optimization and reliable calibration. This performance reflects a strong inductive bias, where the convolutional filters effectively capture short-range temporal patterns and local correlations that recur across the long sensor traces. Given that the dataset consists of 987 batches, each with 5,000 time steps and 12 features, such localized patterns provide most of the predictive signal, making the CNN well suited to the task.

The BiLSTM demonstrated higher MAE, noisier learning dynamics, and broader residual distributions. Although LSTMs are theoretically capable of modeling long-range dependencies, the very long sequence length (5,000 steps) and the moderate dataset size make recurrent optimization challenging. Gradient propagation over thousands of time steps leads to unstable updates and higher variance, while the underlying temporal structure appears dominated by local, rather than long-horizon, dependencies. As a result, the BiLSTM offers no clear advantage over the CNN and incurs substantially higher computational cost.

The hybrid CNN+BiLSTM model performed the worst overall, exhibiting early training–validation divergence, systematic underestimation, and a collapse to constant predictions for the total impurity target. This behavior suggests that the recurrent layer introduces excess capacity relative to the amount of data available and amplifies noise in the harder-to-learn impurity target. In this setting, recurrence does not extract additional useful temporal information beyond what the CNN already captures, but instead destabilizes training.

5.3. ML vs DL comparison

For `total_waste`, the 1D CNN delivers noticeably better predictive performance than Random Forest or XGBoost, with lower validation MAE and tighter residual distributions. Total waste is strongly influenced by smooth temporal trends in fill depth, mixing force, and related sensor traces. CNNs excel at detecting these continuous temporal patterns, whereas the traditional ML models operate on aggregated batch-level features (e.g., means, standard deviations, startup statistics). Compressing each batch into a single feature vector inevitably discards temporal structure, so CNN can exploit information than Random Forest and XGBoost.

Although the CNN outperforms the ML baselines for `total_waste`, the improvement is incremental, and the BiLSTM and hybrid CNN+BiLSTM models perform worse than Random Forest. `Total impurities` depends on mechanical variability, compression-force irregularity, and subtle interactions that may not appear directly in the 12 raw time-series channels used by the DL models. Since the `Total impurities` target is relatively noisy and weakly correlated with process variables, DL does not gain much from temporal pattern extraction. In other words, DL helps only when there are meaningful temporal structures. Impurities lack such structure, so CNN advantages are limited. Therefore, DL does not universally outperform traditional ML for this dataset. Its advantage appears only when the underlying process has strong sequential structure.

In other words, DL provides a clear advantage when the underlying process exhibits strong, learnable sequential structure (as in total waste), but offers limited benefit when the signal is diffuse or largely encoded in higher level summary statistics (as appears to be the case for total impurities). Traditional ML remains highly competitive when interpretability, process understanding, and rapid experimentation are priorities, whereas the CNN-based DL model is most useful when prediction accuracy for a temporally structured target is the primary goal. The main differences between the two approaches can be describe in Table 5.

Overall, the 1D CNN is the strongest model for predicting `total_waste`, reflecting the importance of temporal structure in total waste. Traditional ML methods, especially Random Forest, remain valuable for interpreting process behavior and for predicting `Total impurities`, where the signal is weaker and less temporally structured. Together, these results highlight that the most effective modeling strategy depends on the nature of the target variable rather than a one-size-fits-all approach.

	Traditional ML	**DL (1D CNN)**
Input Type	Aggregated batch-level features	Raw multivariate time-series
Feature Engineering	Extensive (aggregations, domain features, DBSCAN filtering)	Minimal (scaling only)
Generalization	Moderate; balanced across both targets	Strong for `total_waste`; limited gain for `Total impurities`
Interpretability	High (SHAP, permutation importance)	Limited; mainly qualitative via plots
Training Time	Low	Higher (long sequences, larger models)
Overfitting Risk	Low–moderate	Medium (especially for recurrent/hybrid models)

Table 5: Comparison of traditional ML and DL (CNN) approaches.

6. Conclusion

This work developed and evaluated an predictive modeling pipeline for estimating total waste and total impurities in a commercial tablet-manufacturing process. By comparing feature-engineered traditional ML models (Random Forest, XGBoost) and sequence-based DL architectures (1D CNN, BiLSTM, CNN+BiLSTM), we identified which methods are most effective for different types of process outputs.

For batch-level models, Random Forest emerged as the most balanced approach. It achieved competitive R^2 for `total_waste` and clearly outperformed XGBoost for `Total impurities`, while yielding stable SHAP attributions. Total waste was governed primarily by fill geometry and mixing-force dynamics, whereas total impurities were driven by more diffuse mechanical variability, particularly compression force and ejection behavior.

For time-series models, the 1D CNN consistently delivered the lowest validation MAE and the tightest residual distributions for `total_waste`. This confirms that local temporal patterns in sensor traces carry substantial information about total waste. In contrast, the BiLSTM and CNN+BiLSTM models offered no advantage and, in some cases, degraded performance due to optimization difficulty on very long sequences and limited data. Taken together, these findings suggest that, for the manufacturing time-series considered here, localized temporal patterns play a more important role than long-range dependencies, favoring convolutional architectures over BiLSTMs, which may overfit when modeling long, noisy sequences. To confirm this conclusion in future work, we can construct many convolutional models with increasing temporal windows and examining whether incorporating longer time spans improves predictive performance. If accuracy saturates for short windows, this would provide direct evidence that local temporal structure is most informative. For `Total impurities`, all DL architectures provided only modest gains over traditional ML, consistent with the weak and distributed signal identified by the feature-importance analysis.

Taken together, these results highlight that modeling strategy should be chosen based on the nature of the target: DL (particularly 1D CNNs) is most beneficial when the outcome is tightly linked to rich temporal structure, whereas traditional ML remains highly effective when the signal is weak or primarily encoded in summary statistics and when interpretability is a priority.

This study has several limitations. The dataset size is modest, especially relative to the sequence length (5,000 time steps), the impurity signal is noisy and only weakly explained by the available variables, and DL hyperparameters and architectures were explored only within a constrained design space. CNN interpretability also remains challenging without specialized visualization techniques. Future efforts should focus on expanding the dataset, integrating domain-specific features, exploring more advanced architectures such as Transformers, and incorporating multitask or physics approaches. Together, these directions can further strengthen predictive performance and improve process understanding in pharmaceutical manufacturing.

[1] Janja Žagar and Jurij Mihelič. Big data collection in pharmaceutical manufacturing and its use for product quality predictions. *Scientific Data*, 9:99, 2022.

[2] Janja Žagar. Cholessterol-lowering drug process and quality data. *figshare*, 2021.

[3] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, volume 96, pages 226–231, 1996.

[4] Jörg Sander, Martin Ester, Hans-Peter Kriegel, and Xiaowei Xu. Density-based clustering in spatial databases: The algorithm gdbscan and its applications. *Data Mining and Knowledge Discovery*, 2(2):169–194, 1998.

[5] Hervé Abdi and Lynne J Williams. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics*, 2(4):433–459, 2010.

[6] Minghua Chen, Qunying Liu, Shuheng Chen, Yicen Liu, Chang-Hua Zhang, and Ruihua Liu. Xgboost-based algorithm interpretation and application on post-fault transient stability status prediction of power system. *IEEE Access*, 7:13149–13158, 2019.

[7] Edoardo Mosca, Ferenc Szigeti, Stella Tragianni, Daniel Gallagher, and Georg Groh. Shap-based explanation methods: a review for nlp interpretability. In *Proceedings of the*

29th international conference on computational linguistics, pages 4593–4603, 2022.

7. Appendix

7.1. Contribution Table

UNI	kk3764	fnz2101	td2849
Last name	Kim	Zou	Do
Fraction of total contribution	1/3	1/3	1/3
What I did 1	Data preprocessing, non-linear models in ML model	Selection of design variables in ML model	Global interpretation in ML model
What I did 2	Data preprocessing + Create CNN model	Create BiLSTM and CNN+BiLSTM models	Create CNN model
What I did 3	Write ML part (methods, results)	Write DL part (methods, results)	Write the rest (intro, discussion, conclsion, abstract)
What I did 4	Add code ownership	Add code descriptions	Train model + Plot DL figures
What I did 5	Review code + report	Review code + report	Review code + report

Table 6: Individual Student Contributions in Fractions

7.2. ML plots

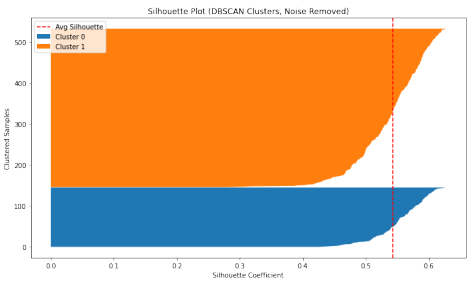

Figure 14: Silhouette plot for DBSCAN clustering after noise removal. Each bar represents the silhouette score of a sample grouped by its cluster assignment.

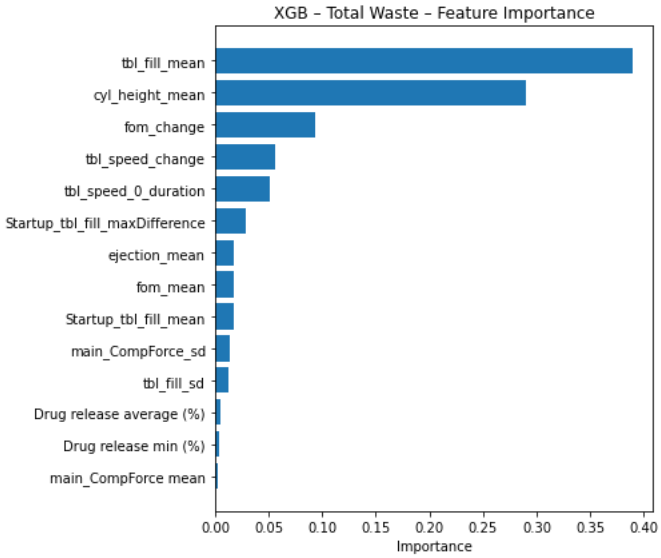

Figure 15: Feature importance for `total_waste` using XGBoost.

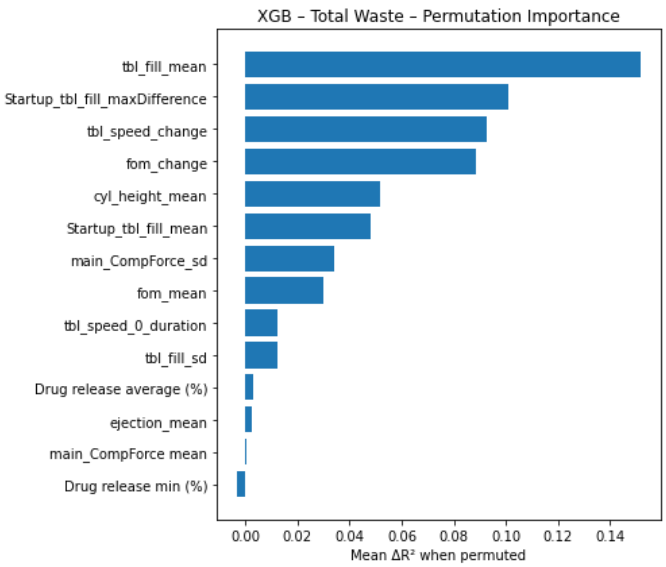

Figure 16: Permutation importance for `total_waste` using XGBoost.

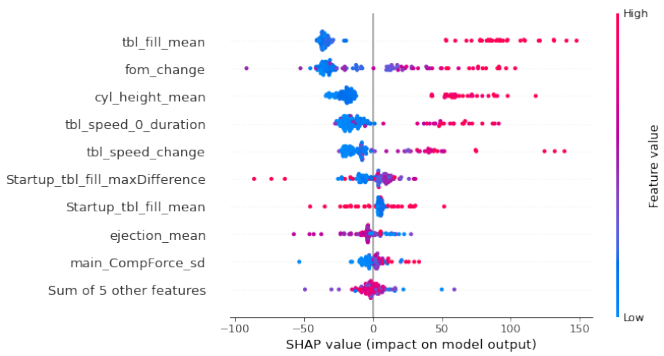

Figure 17: SHAP plot for `total_waste` using XGBoost.

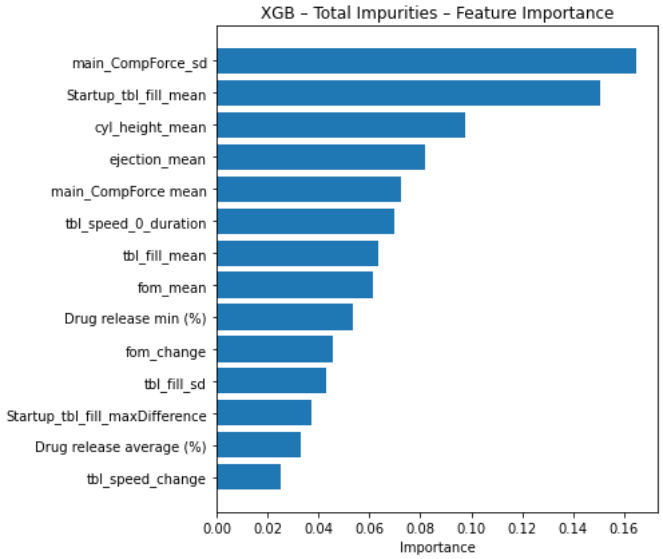

Figure 18: Feature importance for `Total impurities` using XGBoost.

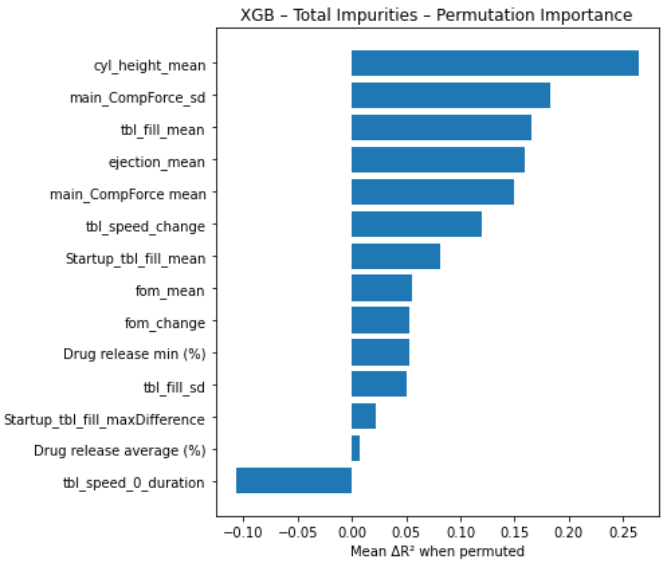

Figure 19: Permutation importance for `Total impurities` using XGBoost.

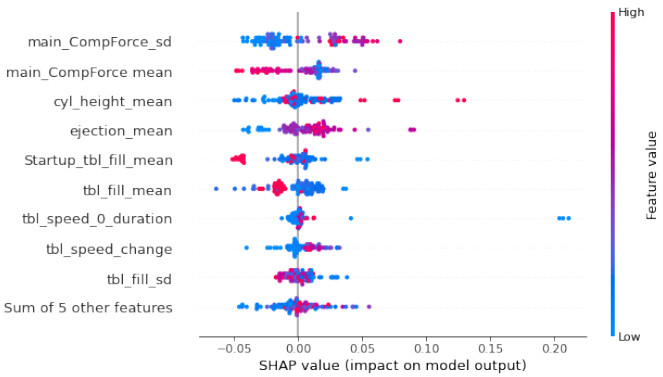

Figure 20: SHAP plot for `Total impurities` using XGBoost.